ACIT 3855 – Service Oriented Architectures – Winter 2026

Microservices – Concepts

Martin Fowler is a Software Architect at a software consulting company called Thoughtworks and wrote (along with James Lewis) some of the early articles on a Microservices Architecture (MSA).

High level definition of an MSA:

In short, the microservice architectural style is an approach to developing a single application as a **suite of small services**, each **running in its own process** and communicating with lightweight mechanisms, often an HTTP resource API. These services are **built around business capabilities** and **independently deployable** by fully automated deployment machinery. There is a **bare minimum of centralized management** of these services, which may be written in different programming languages and use different data storage technologies.
-- James Lewis and Martin Fowler (2014)

Benefits and Costs of an MSA:

Microservices provide benefits…

- Strong Module Boundaries: Microservices reinforce modular structure, which is particularly important for larger teams.
- Independent Deployment: Simple services are easier to deploy, and since they are autonomous, are less likely to cause system failures when they go wrong.
- Technology Diversity: With microservices you can mix multiple languages, development frameworks and data-storage technologies.

…but come with costs

- Distribution: Distributed systems are harder to program, since remote calls are slow and are always at risk of failure.
- Eventual Consistency: Maintaining strong consistency is extremely difficult for a distributed system, which means everyone has to manage eventual consistency.
- Operational Complexity: You need a mature operations team to manage lots of services, which are being redeployed regularly.

(from Microservice Trade-Offs)

Reference: https://martinfowler.com/microservices/

Also read the characteristics of a Microservices Architecture defined here:

https://martinfowler.com/articles/microservices.html#CharacteristicsOfAMicroserviceArchitecture

Lab Preparation

For the Week 2 lab, you will be implementing your first microservice based on your OpenAPI specification from Week 1.

We will be using the Python Connexion library, which is built upon the Flask framework you used in ACIT 2515. The Connexion documentation is located here: https://connexion.readthedocs.io/en/latest/

The most basic Connexion application looks as follows. It is typically put into a file called app.py:

```
import connexion
from connexion import NoContent

# Your functions here to handle your endpoints

app = connexion.FlaskApp(__name__, specification_dir='')
app.add_api("openapi.yml",
            strict_validation=True,
            validate_responses=True)

if __name__ == "__main__":
    app.run(port=8080)
```

Directory to look for OpenAPI Specifications. Empty means look in current directory.

OpenAPI Specification File to Use

Validate Response Messages from Endpoints against OpenAPI Specification

Validate Request Messages and Parameters for Endpoints against OpenAPI Specification

Port on which to run the application.

How do we explicitly route an endpoint from your OpenAPI specification to a function in your Connexion application? For example from the Lab 1 example OpenAPI specification there was an endpoint for reporting a batch of temperature reading readings from a weather station:

```
paths:
  /weather/temperature:
    post:
      summary: Reports a batch of temperature readings
      operationId: app.report_temperature_readings
      description: Adds a new batch of temp readings to the system
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/TemperatureReadingBatch'
        description: Reading item to add
      responses:
        '201':
          description: item created
        '400':
          description: 'invalid input, object invalid'
```

Endpoint for Temperature Reading Batch

operationId: Corresponds to the name of the module (i.e., app) and function (i.e., report_temperature_readings) in your Python app that will handle this endpoint.

References the schema that defined the request message

All possible response codes from this endpoint.

So in our corresponding app.py Connexion module we can create a function to handle the POST method on the /weather/temperature endpoint on your server:

```
def report_temperature_readings(body):
    """ Receives a temperature reading
        Batch event """


    # Handle the request here


    return NoContent, 201
```

report_temperature_readings matches the function name in the **operationId** of the OpenAPI specification.

body – contains the request body already deserialized to a Python dictionary or list.

There are two return values, the response message and the response code. The response message is automatically serialized to JSON, but use **NoContent** when there is no response message returned.

So the overall stubbed out app.py file for the sample OpenAPI specification from Lab 1 would look like:

```
import connexion
from connexion import NoContent

def report_temperature_readings(body):
    """ Receives a temperature reading batch event """

    # Implement Here

    return NoContent, 201


def report_precipitation_readings(body):
    """ Receives a precipitation reading batch event """

    # Implement Here

    return NoContent, 201


app = connexion.FlaskApp(__name__, specification_dir='')
app.add_api("openapi.yml",
            strict_validation=True,
            validate_responses=True)

if __name__ == "__main__":
    app.run(port=8080)
```